

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский университет ИТМО**

**Отчет по лабораторной работе №5
«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ в PostgreSQL»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Буй Тхук Хуен
Факультет прикладной информатики - **Группа** К3239
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2023
Преподаватель Говорова Марина Михайловна
Белов Александр Олегович

Санкт-Петербург
2026 г.

1. Цель работы

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

2. Практическое задание

- Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры.
- Создать триггеры для индивидуальной БД согласно варианту: 7 триггеров по ИЗ.

3. Выполнение

3.1 Процедуры

Создать хранимые процедуры.

- Для снижения цен на книги, которые находятся на базе в количестве, превышающем 1000 штук.
- Для удаления заказов, имеющих статус “отменен”.
- Для ввода нового заказа.

Процедура 1: Для снижения цен на книги, которые находятся на базе в количестве, превышающем 1000 штук.

Процедура принимает один параметр `p_percent`, который определяет величину скидки. Изменение применяется только к тем книгам, у которых значение `stock_quantity` больше 1000, то есть на складе находится большое количество экземпляров.

→ Это позволяет автоматически снижать цену на товары с избыточным запасом для повышения продаж.

Внутри процедуры используется команда `UPDATE`, которая изменяет значение поля `retail_price`. Новая цена вычисляется по формуле уменьшения текущей стоимости на указанный процент. После успешного выполнения процедуры выводится сообщение `Book prices updated` с помощью оператора `RAISE NOTICE`.

```
publishing_book=# create or replace procedure reduce_book_price(  
publishing_book(# in p_percent numeric  
publishing_book(# )  
publishing_book=# language plpgsql  
publishing_book=# as $$  
publishing_book$# begin  
publishing_book$# update book  
publishing_book$# set retail_price = retail_price - (retail_price * p_percent / 100)  
publishing_book$# where stock_quantity > 1000;  
publishing_book$# raise notice 'book prices updated';  
publishing_book$# end;  
publishing_book$# $$;  
CREATE PROCEDURE
```

Перед тестированием процедуры посмотреть список книг и их цены до изменения

```
publishing_book=# select isbn,title,stock_quantity,retail_price from book where stock_quantity > 1000;
```

isbn	title	stock_quantity	retail_price
ISBN-6	Book Title 6	1207	641.37
ISBN-10	Book Title 10	1179	134.21
ISBN-77	Book Title 77	1259	342.82
ISBN-134	Book Title 134	1500	337.17
ISBN-100	Book Title 100	1500	835.14
ISBN-161	Book Title 161	1158	298.95
ISBN-163	Book Title 163	1326	610.27
ISBN-174	Book Title 174	1294	763.02
ISBN-183	Book Title 183	1326	409.19
ISBN-246	Book Title 246	1278	335.35
ISBN-258	Book Title 258	1150	198.88
ISBN-272	Book Title 272	1238	443.34
ISBN-287	Book Title 287	1389	182.32
ISBN-305	Book Title 305	1097	940.28
ISBN-310	Book Title 310	1239	800.57
ISBN-409	Book Title 409	1309	727.40
ISBN-426	Book Title 426	1107	377.50
ISBN-431	Book Title 431	1110	603.90
ISBN-432	Book Title 432	1222	162.69
ISBN-458	Book Title 458	1102	557.89
ISBN-217	Book Title 217	1314	343.2330
ISBN-338	Book Title 338	1093	1610.8455

(22 rows)

Затем процедура была вызвана командой: CALL reduce_book_price(15);

После выполнения процедуры был повторно выполнен тот же запрос для проверки результата. В результате тестирования было видно, что у всех книг с количеством экземпляров больше 1000 цена уменьшилась на 15%.

```
publishing_book=# call reduce_book_price(15);
NOTICE: book prices updated
CALL
publishing_book=# select isbn,title,stock_quantity,retail_price from book where stock_quantity > 1000;
```

isbn	title	stock_quantity	retail_price
ISBN-6	Book Title 6	1207	545.1645000000000000
ISBN-10	Book Title 10	1179	114.0785000000000000
ISBN-77	Book Title 77	1259	291.3970000000000000
ISBN-134	Book Title 134	1500	286.5945000000000000
ISBN-100	Book Title 100	1500	709.8690000000000000
ISBN-161	Book Title 161	1158	254.1075000000000000
ISBN-163	Book Title 163	1326	518.7295000000000000
ISBN-174	Book Title 174	1294	648.5670000000000000
ISBN-183	Book Title 183	1326	347.8115000000000000
ISBN-246	Book Title 246	1278	285.0475000000000000
ISBN-258	Book Title 258	1150	169.0480000000000000
ISBN-272	Book Title 272	1238	376.8390000000000000
ISBN-287	Book Title 287	1389	154.9720000000000000
ISBN-305	Book Title 305	1097	799.2380000000000000
ISBN-310	Book Title 310	1239	680.4845000000000000
ISBN-409	Book Title 409	1309	618.2900000000000000
ISBN-426	Book Title 426	1107	320.8750000000000000
ISBN-431	Book Title 431	1110	513.3150000000000000
ISBN-432	Book Title 432	1222	138.2865000000000000
ISBN-458	Book Title 458	1102	474.2065000000000000
ISBN-217	Book Title 217	1314	291.7480500000000000
ISBN-338	Book Title 338	1093	1369.2186750000000000

(22 rows)

Процедура 2: Для удаления заказов, имеющих статус “отменен”

Вторая процедура delete_cancelled_orders предназначена для удаления заказов со статусом отменен. Такая процедура позволяет автоматически очищать таблицу orders от неактуальных записей.

```

publishing_book=# create or replace procedure delete_cancelled_orders()
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$# begin
publishing_book$# delete from invoice where id_contract in (
publishing_book$# select id_contract from contract where id_order in (
publishing_book$# select id_order from orders where status = 'отменено'
publishing_book$# )
publishing_book$# );
publishing_book$# delete from act where id_contract in (
publishing_book$# select id_contract from contract where id_order in (
publishing_book$# select id_order from orders where status = 'отменено'
publishing_book$# )
publishing_book$# );
publishing_book$# delete from contract where id_order in (
publishing_book$# select id_order from orders where status = 'отменено'
publishing_book$# );
publishing_book$# delete from order_item where id_order in (
publishing_book$# select id_order from orders where status = 'отменено'
publishing_book$# );
publishing_book$# delete from orders where status = 'отменено';
publishing_book$# raise notice 'cancelled orders deleted';
publishing_book$# end;
publishing_book$# $$;
CREATE PROCEDURE

```

Перед запуском процедуры был выполнен запрос. Этот запрос показал все отменённые заказы, существующие в таблице до удаления.

```

publishing_book=# select id_order, status from orders where status = 'отменено';

```

id_order	status
2	отменено
5	отменено
11	отменено
12	отменено
14	отменено
16	отменено
19	отменено
20	отменено
21	отменено
27	отменено
44	отменено
62	отменено
74	отменено
87	отменено
91	отменено
107	отменено
108	отменено
116	отменено
122	отменено
132	отменено
136	отменено

Затем процедура была вызвана командой.

```
publishing_book=# call delete_cancelled_orders();
NOTICE: cancelled orders deleted
CALL
publishing_book=# select * from orders where status = 'отменено';
 id_order | id_manager | id_customer | order_date | status | due_date | total_amount
-----+-----+-----+-----+-----+-----+-----
(0 rows)
```

После выполнения процедуры повторная проверка показала, что записи со статусом отменен были успешно удалены из таблицы.

Процедура 3: Для ввода нового заказа.

Третья процедура `add_new_order` используется для добавления нового заказа в таблицу `orders`. Процедура принимает несколько параметров: идентификатор менеджера, идентификатор клиента, дату заказа, статус заказа, дату выполнения и общую сумму заказа.

Внутри процедуры используется команда `INSERT INTO`, которая добавляет новую запись в таблицу `orders`. Значения всех полей передаются через параметры процедуры. После успешного добавления нового заказа выводится сообщение `New order added`.

```
publishing_book=# create or replace procedure add_new_order(
publishing_book(# in p_manager_id int,
publishing_book(# in p_customer_id int,
publishing_book(# in p_order_date date,
publishing_book(# in p_status text,
publishing_book(# in p_due_date date,
publishing_book(# in p_total_amount numeric
publishing_book(# )
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$$ begin
publishing_book$$ insert into orders(id_manager, id_customer, order_date, status, due_date, total_amount
publishing_book$$ )
publishing_book$$ values(p_manager_id, p_customer_id, p_order_date, p_status, p_due_date, p_total_amount
publishing_book$$ );
publishing_book$$ raise notice 'new order added';
publishing_book$$ end;
publishing_book$$ $$;
CREATE PROCEDURE
```

Для тестирования процедуры была выполнена команда:

```
publishing_book=# call add_new_order(1, 1, current_date, 'получено', current_date + 7, 5000);
NOTICE: new order added
CALL
publishing_book=# select * from orders order by id_order desc limit 5;
 id_order | id_manager | id_customer | order_date | status | due_date | total_amount
-----+-----+-----+-----+-----+-----+-----
      301 |          1 |          1 | 2026-05-16 | получено | 2026-05-23 |          5000
      300 |         23 |         151 | 2026-03-25 | завершено | 2026-05-18 |         8977.17
      299 |         41 |         135 | 2026-03-25 | завершено | 2026-05-21 |         7069.12
      298 |         14 |          58 | 2026-04-15 | завершено | 2026-04-30 |         3127.10
      296 |         34 |          38 | 2026-04-20 | обработано | 2026-05-19 |         8821.40
(5 rows)
```

В результате выполнения в таблицу `orders` был добавлен новый заказ с указанными параметрами. После тестирования было видно, что новая запись успешно

появилась в таблице заказов, а все переданные значения были корректно сохранены в базе данных.

3.2 Триггер

а. Логирование добавления нового клиента

Для реализации автоматического журналирования новых клиентов была создана таблица logs, в которой сохраняется идентификатор клиента и время выполнения операции.

```
publishing_book=# create table logs(  
publishing_book(# id serial primary key,  
publishing_book(# log_text text,  
publishing_book(# created_at timestamp default now()  
publishing_book(# );  
CREATE TABLE
```

После этого была создана функция insert_customer, работающая как trigger function. Данная функция автоматически выполняется после добавления новой записи в таблицу customer. Внутри функции используется команда INSERT, которая добавляет в таблицу logs идентификатор нового клиента через значение NEW.id_customer. Ключевое слово NEW содержит данные новой строки, которая была только что вставлена в таблицу.

```

publishing_book=# create or replace function customer_insert()
publishing_book=# returns trigger
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$$ begin
publishing_book$$ insert into logs(log_text)
publishing_book$$ values('new customer added: ' || new.company_name);
publishing_book$$ return new;
publishing_book$$ end;
publishing_book$$ $$;
CREATE FUNCTION
publishing_book=# create trigger customer_insert
publishing_book=# after insert
publishing_book=# on customer
publishing_book=# for each row
publishing_book=# execute function customer_insert();
CREATE TRIGGER
publishing_book=# insert into customer(company_name, contact_legal_entity, id_tax, phone, address, email)
publishing_book=# values('tech corp', 'test trigger', '9876543210', '+7994403168','sochi', 'testtrigger@gmail.com');
INSERT 0 1
publishing_book=# select * from logs order by id desc;
 id |          log_text          |          created_at
-----+-----+-----
  1 | new customer added: tech corp | 2026-05-16 15:24:53.039469
(1 row)

publishing_book=# insert into customer(company_name, contact_legal_entity, id_tax, phone, address, email)
publishing_book=# values('family1234', 'test trigger3', '9876543211', '+79944033111','moscow', 'testtrigger3@gmail.com');
INSERT 0 1
publishing_book=# select * from logs order by id desc;
 id |          log_text          |          created_at
-----+-----+-----
  2 | new customer added: family1234 | 2026-05-16 15:28:49.155498
  1 | new customer added: tech corp | 2026-05-16 15:24:53.039469
(2 rows)

publishing_book=# |

```

Для автоматического запуска функции был создан триггер `customer_insert`. Он срабатывает после выполнения операции `INSERT` для каждой новой строки таблицы `customer`.

Для проверки работы триггера был выполнен запрос на добавление нового клиента. В результате тестирования было видно, что после добавления клиента в таблице `logs` автоматически появилась новая запись с идентификатором клиента и временем операции. Это подтверждает корректную работу триггера.

b. Автоматическая обработка order item

Функция `process_order_item` была создана для автоматической обработки данных при добавлении новой записи в таблицу `order_item`. Основная задача функции заключается в проверке наличия достаточного количества книг на складе, автоматическом вычислении цены заказа и обновлении остатка товара.

Для автоматического выполнения функции был создан триггер `process_order_item`, который срабатывает перед добавлением новой записи в таблицу `order_item`.

```

publishing_book=# create or replace function process_order_item()
publishing_book=# returns trigger
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$$ declare
publishing_book$$ v_stock int;
publishing_book$$ begin
publishing_book$$ select stock_quantity
publishing_book$$ into v_stock from book where isbn = new.isbn for update;
publishing_book$$ if v_stock < new.quantity then raise exception 'insufficient stock';
publishing_book$$ end if;
publishing_book$$ update book
publishing_book$$ set stock_quantity = stock_quantity - new.quantity
publishing_book$$ where isbn = new.isbn;
publishing_book$$ select retail_price into new.unit_price from book where isbn = new.isbn;
publishing_book$$ new.total_amount := new.quantity * new.unit_price;
publishing_book$$ return new;
publishing_book$$ end;
publishing_book$$ $$;
CREATE FUNCTION
publishing_book=# create trigger process_order_item
publishing_book=# before insert
publishing_book=# on order_item
publishing_book=# for each row
publishing_book=# execute function process_order_item();
CREATE TRIGGER
publishing_book=# select isbn, stock_quantity, retail_price
publishing_book=# from book
publishing_book=# where isbn = 'ISBN-10';
 isbn | stock_quantity | retail_price
-----+-----+-----
ISBN-10 | 1179 | 114.0785000000000000
(1 row)

```

Перед тестированием был выполнен запрос: Результат показал, что книга ISBN-10 имела количество 1179 экземпляров и цену 114.0785.

После этого была добавлена новая запись

```

publishing_book=# insert into order_item(id_order, isbn, quantity)
publishing_book=# values(1, 'ISBN-10', 15);
INSERT 0 1
publishing_book=# select * from order_item where id_order = 1;
 id_order | isbn      | quantity | unit_price | total_amount
-----+-----+-----+-----+-----
1 | ISBN-29 | 4 | 108.25 | 433.00
1 | ISBN-354 | 4 | 108.25 | 433.00
1 | ISBN-385 | 4 | 108.25 | 433.00
1 | ISBN-157 | 4 | 108.25 | 433.00
1 | ISBN-10 | 15 | 114.0785000000000000 | 1711.1775000000000000
(5 rows)

publishing_book=# select isbn, stock_quantity from book where isbn = 'ISBN-10';
 isbn | stock_quantity
-----+-----
ISBN-10 | 1164
(1 row)

```

В результате появилась новая запись для книги ISBN-10, где значение unit_price автоматически стало равно 114.0785, а поле total_amount было вычислено как 1711.1775. И количество экземпляров уменьшилось с 1179 до 1164, то есть на 15 книг.

Дополнительно была выполнена проверка общей суммы заказа:

```
publishing_book=# select id_order, total_amount from orders where id_order = 1;
 id_order |      total_amount
-----+-----
      1 | 3443.1775000000000000
(1 row)
```

Значение total_amount заказа стало равно 3443.1775, что подтверждает корректную обработку данных после добавления нового order item.

с. Запрет нескольких main editor

Функция check_main_editor была создана для контроля бизнес-правила, согласно которому у одного technical assignment может быть только один главный редактор (main_editor = true).

Функция выполняется перед добавлением или изменением записи в таблице editing_ta. Внутри функции используется проверка через оператор EXISTS. Если новая запись имеет значение main_editor = true, функция ищет другую запись с тем же id_ta, у которой также установлен главный редактор. При нахождении такой записи генерируется ошибка с помощью команды RAISE EXCEPTION.

Для автоматической проверки был создан триггер trg_check_main_editor, который срабатывает перед операциями INSERT и UPDATE для каждой строки таблицы editing_ta.

```
publishing_book=# create or replace function check_main_editor()
publishing_book=# returns trigger
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$$ begin
publishing_book$$ if new.main_editor = 'true' then
publishing_book$$ if exists(select 1 from editing_ta where id_ta = new.id_ta and main_editor = 'true'
and id_editing <> new.id_editing) then
publishing_book$$ raise exception 'main editor already exists';
publishing_book$$ end if;
publishing_book$$ end if;
publishing_book$$ return new;
publishing_book$$ end;
publishing_book$$ $$;
CREATE FUNCTION
publishing_book=# create trigger check_main_editor
publishing_book=# before insert or update
publishing_book=# on editing_ta
publishing_book=# for each row
publishing_book=# execute function check_main_editor();
CREATE TRIGGER
publishing_book=# select * from editing_ta where id_ta = 1;
 id_editing | id_ta | id_editor | participation_start | order_editor | main_editor |      comment
-----+-----+-----+-----+-----+-----+-----
      1 |    1 |      14 | 2025-05-17         |           1 |      true   | lead editor
      2 |    1 |       8 | 2025-12-02         |           2 |     false   | review editor
(2 rows)
```

Для проверки корректного случая был выполнен запрос:

```
publishing_book=# insert into editing_ta(id_ta, id_editor, id_editing, participation_start, order_edit
or, main_editor, comment)
publishing_book=# values(1, 20, 1001, current_date, 3, false, 'review editor');
INSERT 0 1
```

```
publishing_book=# select * from editing_ta where id_ta = 1;
```

id_editing	id_ta	id_editor	participation_start	order_editor	main_editor	comment
1	1	14	2025-05-17	1	true	lead editor
2	1	8	2025-12-02	2	false	review editor
1001	1	20	2026-05-17	3	false	review editor

(3 rows)

В результате запись была успешно добавлена, так как значение `main_editor` равно `false`.

После этого был выполнен тест с ошибкой:

```
publishing_book=# insert into editing_ta(id_ta, id_editor, id_editing, participation_start, order_editor, main_editor, comment)
publishing_book=# values(1, 25, 1002, current_date, 4, true, 'add main editor');
ERROR: main editor already exists
CONTEXT: PL/pgSQL function check_main_editor() line 5 at RAISE
```

Во время выполнения триггер обнаружил, что для данного `id_ta` уже существует главный редактор.

```
publishing_book=# update editing_ta set main_editor = true where id_editing = 2;
ERROR: main editor already exists
CONTEXT: PL/pgSQL function check_main_editor() line 5 at RAISE
```

Аналогичным образом это относится и к случаю повышения должности редактора до главного редактора.

d. Аудит изменения `circulation`

Для контроля изменений количества экземпляров была создана таблица `circulation_audit`. В этой таблице сохраняются идентификатор записи `circulation`, старое значение количества, новое значение количества и время изменения.

После этого была создана функция `audit_circulation_change`, предназначенная для автоматического журналирования изменений поля `circulation_quantity`.

Функция сравнивает значения `OLD.circulation_quantity` и `NEW.circulation_quantity`. Если значения отличаются, в таблицу `circulation_audit` добавляется новая запись со старым и новым количеством экземпляров. Если значение не изменилось, запись в журнал не создаётся.

Для автоматического запуска функции был создан триггер `trg_audit_circulation`, который выполняется после обновления таблицы `circulation`.

```
publishing_book=# create table circulation_audit(
publishing_book=# id serial primary key,
publishing_book=# id_circulation int,
publishing_book=# old_quantity int,
publishing_book=# new_quantity int,
publishing_book=# changed_at timestamp default now()
publishing_book=# );
CREATE TABLE
publishing_book=# create or replace function audit_circulation_change()
publishing_book=# returns trigger
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$$ begin
publishing_book$$$ if old.circulation_quantity <> new.circulation_quantity then
publishing_book$$$ insert into circulation_audit(id_circulation, old_quantity, new_quantity)
publishing_book$$$ values(old.id_circulation, old.circulation_quantity, new.circulation_quantity);
publishing_book$$$ end if;
publishing_book$$$ return new;
publishing_book$$$ end;
publishing_book$$$ $$;
CREATE FUNCTION
publishing_book=# create trigger audit_circulation
publishing_book=# after update
publishing_book=# on circulation
publishing_book=# for each row
publishing_book=# execute function audit_circulation_change();
CREATE TRIGGER
```

Перед тестированием была выполнена проверка текущих данных. После этого было выполнено изменение количества экземпляров. В результате появилась новая запись, содержащая старое значение 3964 и новое значение 3000

```
publishing_book=# select * from circulation limit 5;
 id_circulation | isbn | print_type | circulation_quantity | circulation_date | notes
-----+-----+-----+-----+-----+-----
          1 | ISBN-1 | дополнительный | 3964 | 2023-03-03 | 
          2 | ISBN-2 | оригинальный | 3866 | 2024-08-11 | reprint
          3 | ISBN-3 | оригинальный | 1189 | 2026-02-25 | 
          4 | ISBN-4 | дополнительный | 2069 | 2024-03-08 | 
          5 | ISBN-5 | дополнительный | 3238 | 2024-03-10 | 
(5 rows)

publishing_book=# update circulation set circulation_quantity = 3000 where id_circulation = 1;
UPDATE 1
publishing_book=# select * from circulation_audit;
 id | id_circulation | old_quantity | new_quantity | changed_at
-----+-----+-----+-----+-----
   1 |          1 | 3964 | 3000 | 2026-05-18 00:10:40.640716
(1 row)
```

После этого был выполнен повторный запрос:

```
publishing_book=# update circulation set circulation_quantity = 1189 where id_circulation = 3;
UPDATE 1
publishing_book=# select * from circulation_audit;
 id | id_circulation | old_quantity | new_quantity | changed_at
-----+-----+-----+-----+-----
   1 |          1 | 3964 | 3000 | 2026-05-18 00:10:40.640716
(1 row)
```

Так как значение не изменилось, функция не добавила новую запись в таблицу аудита. Повторная проверка таблицы `circulation_audit` показала отсутствие новых строк, что подтверждает правильную работу условия внутри триггера.

е. Запрет удаления книги, участвующей в заказах

Функция `prevent_book_delete` была создана для защиты данных в таблице `book`. Её основная задача заключается в предотвращении полного удаления книги, если она уже использовалась в таблице `order_item`, то есть участвовала хотя бы в одном заказе.

Функция работает как `trigger function` и выполняется перед удалением записи из таблицы `book`. Внутри функции сначала выполняется проверка с помощью оператора `EXISTS`. Система ищет наличие записей в таблице `order_item`, где ISBN книги совпадает со значением `OLD.isbn`. Ключевое слово `OLD` содержит данные строки, которая должна быть удалена.

Если книга уже присутствует в заказах, физическое удаление не выполняется. Вместо этого функция изменяет данные книги: значение `stock_quantity` устанавливается равным 1, а к названию книги автоматически добавляется текст `(archived)`. После этого функция возвращает `NULL`, что отменяет операцию удаления.

Если книга не используется ни в одном заказе, функция возвращает `OLD`, и удаление выполняется обычным образом.

```
publishing_book=# create or replace function prevent_book_delete()
publishing_book=# returns trigger
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$# begin
publishing_book$# if exists(select 1 from order_item where isbn = old.isbn) then
publishing_book$# update book set stock_quantity = 1, title = title || ' (archived)' where isbn = old.
isbn;
publishing_book$# return null;
publishing_book$# end if;
publishing_book$# return old;
publishing_book$# end;
publishing_book$# $$;
CREATE FUNCTION
publishing_book=# create trigger prevent_book_delete
publishing_book=# before delete
publishing_book=# on book
publishing_book=# for each row
publishing_book=# execute function prevent_book_delete();
CREATE TRIGGER
```

Для автоматического запуска функции был создан триггер `prevent_book_delete`, который срабатывает перед выполнением операции `DELETE` для каждой строки таблицы `book`.

Перед тестированием была выполнена проверка данных книги: книга ISBN-100 имела название `Book Title 100` и количество экземпляров 1500.

```
publishing_book=# select isbn, title, stock_quantity from book where isbn = 'ISBN-100';
 isbn | title | stock_quantity
-----+-----+-----
ISBN-100 | Book Title 100 | 1500
(1 row)

publishing_book=# delete from book where isbn = 'ISBN-100';
DELETE 0
publishing_book=# select isbn, title, stock_quantity from book where isbn = 'ISBN-100';
 isbn | title | stock_quantity
-----+-----+-----
ISBN-100 | Book Title 100 (archived) | 1
(1 row)
```

После этого была выполнена попытка удаления книги: В результате PostgreSQL вывел сообщение: `DELETE 0` → Это означает, что физическое удаление строки не

произошло. Затем была выполнена повторная проверка: книга осталась в таблице, однако её название изменилось на Book Title 100 (archived), а количество экземпляров было изменено. Это подтверждает, что триггер успешно предотвратил удаление книги, которая уже использовалась в заказах.

f. Ограничение слишком большой скидки

Функция `prevent_extreme_discount` была создана для защиты данных о стоимости книг и предотвращения слишком сильного снижения цены. Основная задача функции заключается в запрете уменьшения значения `retail_price` более чем на 50% от предыдущей цены.

Функция выполняется перед обновлением записей таблицы `book`. Внутри функции сравниваются значения `NEW.retail_price` и `OLD.retail_price`. Ключевое слово `OLD` содержит старое значение цены, а `NEW` — новое значение, которое пользователь пытается установить.

Если новая цена оказывается меньше половины предыдущей стоимости, функция генерирует ошибку. После возникновения ошибки операция обновления автоматически отменяется. Если условие не нарушено, функция возвращает `NEW`, и изменение цены выполняется успешно.

```
publishing_book=# create or replace function prevent_extreme_discount()
publishing_book=# returns trigger
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$# begin
publishing_book$# if new.retail_price < old.retail_price * 0.5 then
publishing_book$# raise exception 'discount exceeds 50 percent';
publishing_book$# end if;
publishing_book$# return new;
publishing_book$# end;
publishing_book$# $$;
CREATE FUNCTION
publishing_book=# create trigger prevent_extreme_discount
publishing_book=# before update
publishing_book=# on book
publishing_book=# for each row
publishing_book=# execute function prevent_extreme_discount();
CREATE TRIGGER
```

Для автоматического запуска функции был создан триггер `prevent_extreme_discount`, который срабатывает перед выполнением операции `UPDATE` в таблице `book` для каждой строки.

Перед тестированием были выполнены запросы для просмотра текущих цен книг.

```
publishing_book=# select isbn, retail_price from book limit 5;
```

isbn	retail_price
ISBN-1	577.36
ISBN-3	133.69
ISBN-4	592.87
ISBN-5	335.41
ISBN-7	742.75

```
(5 rows)
```

Также отдельно были проверены некоторые книги:

```
publishing_book=# update book set retail_price = 600 where isbn = 'ISBN-1';
UPDATE 1
publishing_book=# update book set retail_price = 300 where isbn = 'ISBN-7';
ERROR: discount exceeds 50 percent
CONTEXT: PL/pgSQL function prevent_extreme_discount() line 4 at RAISE
publishing_book=# update book set retail_price = 300 where isbn = 'ISBN-5';
UPDATE 1
```

```
publishing_book=# select isbn, retail_price from book where isbn = 'ISBN-1';
```

isbn	retail_price
ISBN-1	600

```
(1 row)
```

```
publishing_book=# select isbn, retail_price from book where isbn = 'ISBN-7';
```

isbn	retail_price
ISBN-7	742.75

```
(1 row)
```

```
publishing_book=# select isbn, retail_price from book where isbn = 'ISBN-5';
```

isbn	retail_price
ISBN-5	300

```
(1 row)
```

Если попытаться уменьшить цену книги ISBN-1 с 600 до значения меньше 300, триггер остановит выполнение операции и выдаст сообщение: ERROR: discount exceeds 50 percent

Таким образом, триггер успешно контролирует изменение цен и предотвращает установку слишком больших скидок в системе.

g. Автоматическое создание дополнительного тиража

Функция `auto_reorder_book` была создана для автоматического контроля количества книг на складе. Основная задача функции заключается в создании новой записи о дополнительном тираже, если количество экземпляров книги становится слишком маленьким.

Функция выполняется после обновления записей таблицы `book`. Внутри функции проверяется значение `NEW.stock_quantity`. Если количество экземпляров

книги становится меньше 5, система автоматически создаёт новую запись в таблице circulation.

Для автоматического запуска функции был создан триггер auto_reorder_book, который срабатывает после выполнения операции UPDATE в таблице book для каждой строки.

```
publishing_book=# create or replace function auto_reorder_book()
publishing_book=# returns trigger
publishing_book=# language plpgsql
publishing_book=# as $$
publishing_book$# begin
publishing_book$# if new.stock_quantity < 5 then
publishing_book$# insert into circulation(isbn, print_type, circulation_quantity, circulation_date)
publishing_book$# values(new.isbn, 'дополнительный', 1000, current_date);
publishing_book$# end if;
publishing_book$# return new;
publishing_book$# end;
publishing_book$# $$;
CREATE FUNCTION
publishing_book=# create trigger auto_reorder_book
publishing_book=# after update
publishing_book=# on book
publishing_book=# for each row
publishing_book=# execute function auto_reorder_book();
CREATE TRIGGER
```

Перед тестированием была выполнена проверка количества экземпляров книги: книга ISBN-222 имела 101 экземпляр на складе.

```
publishing_book=# select isbn, stock_quantity from book where isbn = 'ISBN-222';
 isbn | stock_quantity
-----+-----
ISBN-222 |          101
(1 row)
```

После этого было выполнено обновление количества:

```
publishing_book=# update book set stock_quantity = 4 where isbn = 'ISBN-222';
UPDATE 1
publishing_book=# select * from circulation where isbn = 'ISBN-222' order by id_circulation desc;
 id_circulation | isbn | print_type | circulation_quantity | circulation_date | notes
-----+-----+-----+-----+-----+-----
          1004 | ISBN-222 | дополнительный |          1000 | 2026-05-18 |
           722 | ISBN-222 | дополнительный |           717 | 2025-12-28 |
           222 | ISBN-222 | дополнительный |          4794 | 2023-11-22 | reprint
(3 rows)
```

Так как новое значение оказалось меньше 5, триггер автоматически вызвал функцию auto_reorder_book. В результате появилась новая запись в таблице circulation с типом дополнительный, количеством 1000 экземпляров и текущей датой 2026-05-18. Это подтверждает, что триггер успешно обнаружил критически низкий остаток книги и автоматически создал дополнительный тираж.

4. Выводы

В ходе выполнения лабораторной работы были изучены возможности использования процедур, функций и триггеров в PostgreSQL для автоматизации работы базы данных издательской системы. Были разработаны процедуры для изменения цен книг, удаления отменённых заказов и добавления новых заказов. Это позволило автоматизировать типовые операции и упростить обработку данных.

Также были созданы различные триггеры, обеспечивающие контроль целостности данных и автоматическое выполнение бизнес-логики. В системе было реализовано логирование добавления новых клиентов, автоматическое вычисление стоимости заказа и обновление количества книг на складе, контроль наличия только одного главного редактора для одного technical assignment, аудит изменений тиража, защита книг от удаления при наличии связанных заказов, ограничение слишком больших скидок и автоматическое создание дополнительного тиража при низком остатке книг.

Во время тестирования были проверены все основные сценарии работы триггеров и процедур. Результаты показали, что функции корректно выполняют необходимые проверки, автоматически изменяют данные и предотвращают возникновение ошибок и некорректных операций.

Таким образом, лабораторная работа позволила получить практические навыки работы с SQL Shell, триггерами и процедурами PostgreSQL, а также продемонстрировала эффективность использования серверной логики для автоматизации процессов и обеспечения надёжности базы данных.